JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ConcurrentSkipListMap<K,V>

java.lang.Object
  java.util.AbstractMap<K,V>
    java.util.concurrent.ConcurrentSkipListMap<K,V>

Type Parameters:

K - the type of keys maintained by this map

V - the type of mapped values

All Implemented Interfaces:

Serializable, Cloneable, ConcurrentMap<K,V>, ConcurrentNavigableMap<K,V>, Map<K,V>, NavigableMap<K,V>, SequencedMap<K,V>, SortedMap<K,V>

```
public class ConcurrentSkipListMap<K,V>
extends AbstractMap<K,V>
implements ConcurrentNavigableMap<K,V>, Cloneable, Serializable
```

A scalable concurrent ConcurrentNavigableMap implementation. The map is sorted according to the natural ordering of its keys, or by a Comparator provided at map creation time, depending on which constructor is used.

This class implements a concurrent variant of SkipLists providing expected average *log(n)* time cost for the containsKey, get, put and remove operations and their variants. Insertion, removal, update, and access operations safely execute concurrently by multiple threads.

Iterators and spliterators are *weakly consistent*.

Ascending key ordered views and their iterators are faster than descending ones.

All Map.Entry pairs returned by methods in this class and its views represent snapshots of mappings at the time they were produced. They do *not* support the Entry.setValue method. (Note however that it is possible to change mappings in the associated map using put, putIfAbsent, or replace, depending on exactly which effect you need.)

Beware that bulk operations putAll, equals, toArray, containsValue, and clear are *not* guaranteed to be performed atomically. For example, an iterator operating concurrently with a putAll operation might view only some of the added elements.

This class and its views and iterators implement all of the *optional* methods of the Map and Iterator interfaces. Like most other concurrent collections, this class does *not* permit the use of null keys or values because some null return values cannot be reliably distinguished from the absence of elements.

This class is a member of the Java Collections Framework.

Since:

1.6

See Also:

Serialized Form

Nested Class Summary

Nested classes/interfaces declared in class AbstractMap
AbstractMap.SimpleEntry<K,V>, AbstractMap.SimpleImmutableEntry<K,V>

Nested classes/interfaces declared in interface Map
Map.Entry<K,V>

Constructor Summary

Constructors

Constructor	Description
ConcurrentSkipListMap()	Constructs a new, empty map, sorted according to the natural ordering of the keys.
ConcurrentSkipListMap(Comparator<? super K> comparator)	Constructs a new, empty map, sorted according to the specified comparator.
ConcurrentSkipListMap(Map<? extends K, ? extends V> m)	Constructs a new map containing the same mappings as the given map, sorted according to the natural ordering of the keys.
ConcurrentSkipListMap(SortedMap<K, ? extends V> m)	Constructs a new map containing the same mappings and using the same ordering as the specified sorted map.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description

Map.Entry<K,V>	**ceilingEntry**(K key)	Returns a key-value mapping associated with the least key greater than or equal to the given key, or `null` if there is no such entry.
K	**ceilingKey**(K key)	Returns the least key greater than or equal to the given key, or `null` if there is no such key.
void	**clear**()	Removes all of the mappings from this map.
ConcurrentSkipListMa<K,V>	**clone**()	Returns a shallow copy of this `ConcurrentSkipListMap` instance.
Comparator<? super K>	**comparator**()	Returns the comparator used to order the keys in this map, or `null` if this map uses the natural ordering of its keys.
V	**compute**(K key, BiFunction<? super K, ? super V, ? extends V> remappingFunction)	Attempts to compute a mapping for the specified key and its current mapped value (or `null` if there is no current mapping).
V	**computeIfAbsent**(K key, Function<? super K, ? extends V> mappingFunction)	If the specified key is not already associated with a value, attempts to compute its value using the given mapping function and enters it into this map unless `null`.
V	**computeIfPresent**(K key, BiFunction<? super K, ? super V, ? extends V> remappingFunction)	If the value for the specified key is present, attempts to compute a new mapping given the

		key and its current mapped value.
boolean	**containsKey**(Object key)	Returns true if this map contains a mapping for the specified key.
boolean	**containsValue**(Object value)	Returns true if this map maps one or more keys to the specified value.
NavigableSet<K>	**descendingKeySet**()	Returns a reverse order NavigableSet view of the keys contained in this map.
ConcurrentNavigableM<K,V>	**descendingMap**()	Returns a reverse order view of the mappings contained in this map.
Set<Map.Entry<K,V>>	**entrySet**()	Returns a Set view of the mappings contained in this map.
boolean	**equals**(Object o)	Compares the specified object with this map for equality.
Map.Entry<K,V>	**firstEntry**()	Returns a key-value mapping associated with the least key in this map, or null if the map is empty.
K	**firstKey**()	Returns the first (lowest) key currently in this map.
Map.Entry<K,V>	**floorEntry**(K key)	Returns a key-value mapping associated with the greatest key less than or equal to the

		given key, or `null` if there is no such key.
`K`	**`floorKey`**`(K key)`	Returns the greatest key less than or equal to the given key, or `null` if there is no such key.
`V`	**`get`**`(Object key)`	Returns the value to which the specified key is mapped, or `null` if this map contains no mapping for the key.
`V`	**`getOrDefault`**`(Object key, V defaultValue)`	Returns the value to which the specified key is mapped, or the given defaultValue if this map contains no mapping for the key.
`ConcurrentNavigableM` `<K,V>`	**`headMap`**`(K toKey)`	Returns a view of the portion of this map whose keys are strictly less than `toKey`.
`ConcurrentNavigableM` `<K,V>`	**`headMap`**`(K toKey, boolean inclusive)`	Returns a view of the portion of this map whose keys are less than (or equal to, if `inclusive` is true) `toKey`.
`Map.Entry<K,V>`	**`higherEntry`**`(K key)`	Returns a key-value mapping associated with the least key strictly greater than the given key, or `null` if there is no such key.
`K`	**`higherKey`**`(K key)`	Returns the least key strictly greater than the

		given key, or `null` if there is no such key.
`NavigableSet<K>`	**`keySet`**`()`	Returns a `NavigableSet` view of the keys contained in this map.
`Map.Entry<K,V>`	**`lastEntry`**`()`	Returns a key-value mapping associated with the greatest key in this map, or `null` if the map is empty.
`K`	**`lastKey`**`()`	Returns the last (highest) key currently in this map.
`Map.Entry<K,V>`	**`lowerEntry`**`(K key)`	Returns a key-value mapping associated with the greatest key strictly less than the given key, or `null` if there is no such key.
`K`	**`lowerKey`**`(K key)`	Returns the greatest key strictly less than the given key, or `null` if there is no such key.
`V`	**`merge`**`(K key, V value, BiFunction<? super V, ? super V, ? extends V> remappingFunction)`	If the specified key is not already associated with a value, associates it with the given value.
`NavigableSet<K>`	**`navigableKeySet`**`()`	Returns a `NavigableSet` view of the keys contained in this map.
`Map.Entry<K,V>`	**`pollFirstEntry`**`()`	Removes and returns a key-value mapping associated with the least key in this map, or `null` if the map is empty.

Map.Entry<K,V>	**pollLastEntry**()	Removes and returns a key-value mapping associated with the greatest key in this map, or `null` if the map is empty.
V	**put**(K key, V value)	Associates the specified value with the specified key in this map.
V	**putFirst**(K k, V v)	Throws `UnsupportedOperationException`.
V	**putIfAbsent**(K key, V value)	If the specified key is not already associated with a value, associates it with the given value.
V	**putLast**(K k, V v)	Throws `UnsupportedOperationException`.
V	**remove**(Object key)	Removes the mapping for the specified key from this map if present.
boolean	**remove**(Object key, Object value)	Removes the entry for a key only if currently mapped to a given value.
V	**replace**(K key, V value)	Replaces the entry for a key only if currently mapped to some value.
boolean	**replace**(K key, V oldValue, V newValue)	Replaces the entry for a key only if currently mapped to a given value.
ConcurrentNavigableM<K,V>	**subMap**(K fromKey, boolean fromInclusive, K toKey, boolean toInclusive)	Returns a view of the portion of this map whose keys range from `fromKey` to `toKey`.
ConcurrentNavigableM<K,V>	**subMap**(K fromKey, K toKey)	Returns a view of the portion of this map whose keys range from

		fromKey, inclusive, to toKey, exclusive.
ConcurrentNavigableM <K,V>	**tailMap**(K fromKey)	Returns a view of the portion of this map whose keys are greater than or equal to fromKey.
ConcurrentNavigableM <K,V>	**tailMap**(K fromKey, boolean inclusive)	Returns a view of the portion of this map whose keys are greater than (or equal to, if inclusive is true) fromKey.
Collection<V>	**values**()	Returns a Collection view of the values contained in this map.

Methods declared in class AbstractMap

hashCode, isEmpty, putAll, size, toString

Methods declared in class Object

finalize, getClass, notify, notifyAll, wait, wait, wait

Methods declared in interface ConcurrentMap

forEach, replaceAll

Methods declared in interface NavigableMap

reversed

Methods declared in interface SequencedMap

`sequencedEntrySet`, `sequencedKeySet`, `sequencedValues`

Constructor Details

ConcurrentSkipListMap

```
public ConcurrentSkipListMap()
```

Constructs a new, empty map, sorted according to the natural ordering of the keys.

ConcurrentSkipListMap

```
public ConcurrentSkipListMap(Comparator<? super K> comparator)
```

Constructs a new, empty map, sorted according to the specified comparator.

Parameters:

`comparator` - the comparator that will be used to order this map. If `null`, the natural ordering of the keys will be used.

ConcurrentSkipListMap

```
public ConcurrentSkipListMap(Map<? extends K, ? extends V> m)
```

Constructs a new map containing the same mappings as the given map, sorted according to the natural ordering of the keys.

Parameters:

`m` - the map whose mappings are to be placed in this map

Throws:

`ClassCastException` - if the keys in `m` are not `Comparable`, or are not mutually comparable

`NullPointerException` - if the specified map or any of its keys or values are null

ConcurrentSkipListMap

```
public ConcurrentSkipListMap(SortedMap<K, ? extends V> m)
```

Constructs a new map containing the same mappings and using the same ordering as the specified sorted map.

Parameters:

`m` - the sorted map whose mappings are to be placed in this map, and whose comparator is to be used to sort this map

Throws:

`NullPointerException` - if the specified sorted map or any of its keys or values are null

Method Details

clone

```
public ConcurrentSkipListMap<K,V> clone()
```

Returns a shallow copy of this `ConcurrentSkipListMap` instance. (The keys and values themselves are not cloned.)

Overrides:

`clone` in class `AbstractMap<K,V>`

Returns:

a shallow copy of this map

See Also:

`Cloneable`

containsKey

```
public boolean containsKey(Object key)
```

Returns `true` if this map contains a mapping for the specified key.

Specified by:

`containsKey` in interface `Map<K,V>`

Overrides:

`containsKey` in class `AbstractMap<K,V>`

Parameters:

`key` - key whose presence in this map is to be tested

Returns:

`true` if this map contains a mapping for the specified key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key is null

get

```
public V get(Object key)
```

Returns the value to which the specified key is mapped, or `null` if this map contains no mapping for the key.

More formally, if this map contains a mapping from a key `k` to a value `v` such that `key` compares equal to `k` according to the map's ordering, then this method returns `v`; otherwise it returns `null`. (There can be at most one such mapping.)

Specified by:

`get` in interface `Map<K,V>`

Overrides:

`get` in class `AbstractMap<K,V>`

Parameters:

`key` - the key whose associated value is to be returned

Returns:

the value to which the specified key is mapped, or `null` if this map contains no mapping for the key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key is null

getOrDefault

```
public V getOrDefault(Object key,
                      V defaultValue)
```

Returns the value to which the specified key is mapped, or the given defaultValue if this map contains no mapping for the key.

Specified by:

`getOrDefault` in interface `ConcurrentMap<K,V>`

Specified by:

`getOrDefault` in interface `Map<K,V>`

Parameters:

`key` - the key

`defaultValue` - the value to return if this map contains no mapping for the given key

Returns:

the mapping for the key, if present; else the defaultValue

Throws:

`NullPointerException` - if the specified key is null

Since:

1.8

put

```
public V put(K key,
             V value)
```

Associates the specified value with the specified key in this map. If the map previously contained a mapping for the key, the old value is replaced.

Specified by:

`put` in interface `Map<K,V>`

Overrides:

`put` in class `AbstractMap<K,V>`

Parameters:

`key` - key with which the specified value is to be associated

`value` - value to be associated with the specified key

Returns:

the previous value associated with the specified key, or `null` if there was no mapping for the key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key or value is null

remove

```
public V remove(Object key)
```

Removes the mapping for the specified key from this map if present.

Specified by:

`remove` in interface `Map<K,V>`

Overrides:

`remove` in class `AbstractMap<K,V>`

Parameters:

`key` - key for which mapping should be removed

Returns:

the previous value associated with the specified key, or `null` if there was no mapping for the key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key is null

containsValue

```
public boolean containsValue(Object value)
```

Returns `true` if this map maps one or more keys to the specified value. This operation requires time linear in the map size. Additionally, it is possible for the map to change during execution of this method, in which case the returned result may be inaccurate.

Specified by:

`containsValue` in interface `Map<K,V>`

Overrides:

`containsValue` in class `AbstractMap<K,V>`

Parameters:

`value` - value whose presence in this map is to be tested

Returns:

`true` if a mapping to `value` exists; `false` otherwise

Throws:

`NullPointerException` - if the specified value is null

clear

```
public void clear()
```

Removes all of the mappings from this map.

Specified by:

`clear` in interface `Map<K,V>`

Overrides:

`clear` in class `AbstractMap<K,V>`

computeIfAbsent

```
public V computeIfAbsent
(K key,
 Function<? super K, ? extends V> mappingFunction)
```

If the specified key is not already associated with a value, attempts to compute its value using the given mapping function and enters it into this map unless `null`. The function is *NOT* guaranteed to be applied once atomically only if the value is not present.

Specified by:

`computeIfAbsent` in interface `ConcurrentMap<K,V>`

Specified by:

`computeIfAbsent` in interface `Map<K,V>`

Parameters:

`key` - key with which the specified value is to be associated

`mappingFunction` - the function to compute a value

Returns:

the current (existing or computed) value associated with the specified key, or null if the computed value is null

Throws:

`NullPointerException` - if the specified key is null or the mappingFunction is null

Since:

1.8

computeIfPresent

```
public V computeIfPresent
(K key,
 BiFunction<? super K, ? super V, ? extends V> remappingFunction)
```

If the value for the specified key is present, attempts to compute a new mapping given the key and its current mapped value. The function is *NOT* guaranteed to be applied once atomically.

Specified by:

`computeIfPresent` in interface `ConcurrentMap<K,V>`

Specified by:

`computeIfPresent` in interface `Map<K,V>`

Parameters:

`key` - key with which a value may be associated

`remappingFunction` - the function to compute a value

Returns:

the new value associated with the specified key, or null if none

Throws:

NullPointerException - if the specified key is null or the remappingFunction is null

Since:

1.8

compute

```
public V compute
(K key,
 BiFunction<? super K, ? super V, ? extends V> remappingFunction)
```

Attempts to compute a mapping for the specified key and its current mapped value (or `null` if there is no current mapping). The function is *NOT* guaranteed to be applied once atomically.

Specified by:

compute in interface ConcurrentMap<K,V>

Specified by:

compute in interface Map<K,V>

Parameters:

key - key with which the specified value is to be associated

remappingFunction - the function to compute a value

Returns:

the new value associated with the specified key, or null if none

Throws:

NullPointerException - if the specified key is null or the remappingFunction is null

Since:

1.8

merge

```
public V merge
(K key,
 V value,
 BiFunction<? super V, ? super V, ? extends V> remappingFunction)
```

If the specified key is not already associated with a value, associates it with the given value. Otherwise, replaces the value with the results of the given remapping function, or removes if `null`. The function is *NOT* guaranteed to be applied once atomically.

Specified by:

merge in interface ConcurrentMap<K,V>

Specified by:

`merge` in interface `Map<K,V>`

Parameters:

`key` - key with which the specified value is to be associated

`value` - the value to use if absent

`remappingFunction` - the function to recompute a value if present

Returns:

the new value associated with the specified key, or null if none

Throws:

`NullPointerException` - if the specified key or value is null or the remappingFunction is null

Since:

1.8

keySet

`public NavigableSet<K> keySet()`

Returns a `NavigableSet` view of the keys contained in this map.

The set's iterator returns the keys in ascending order. The set's spliterator additionally reports `Spliterator.CONCURRENT`, `Spliterator.NONNULL`, `Spliterator.SORTED` and `Spliterator.ORDERED`, with an encounter order that is ascending key order.

The spliterator's comparator is `null` if the map's comparator is `null`. Otherwise, the spliterator's comparator is the same as or imposes the same total ordering as the map's comparator.

The set is backed by the map, so changes to the map are reflected in the set, and vice-versa. The set supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`, `Set.remove`, `removeAll`, `retainAll`, and `clear` operations. It does not support the `add` or `addAll` operations.

The view's iterators and spliterators are *weakly consistent*.

This method is equivalent to method `navigableKeySet`.

Specified by:

`keySet` in interface `ConcurrentNavigableMap<K,V>`

Specified by:

`keySet` in interface `Map<K,V>`

Specified by:

keySet in interface SortedMap<K,V>

Overrides:

keySet in class AbstractMap<K,V>

Returns:

a navigable set view of the keys in this map

navigableKeySet

```
public NavigableSet<K> navigableKeySet()
```

Description copied from interface: ConcurrentNavigableMap

Returns a NavigableSet view of the keys contained in this map. The set's iterator returns the keys in ascending order. The set is backed by the map, so changes to the map are reflected in the set, and vice-versa. The set supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`, `Set.remove`, `removeAll`, `retainAll`, and `clear` operations. It does not support the `add` or `addAll` operations.

The view's iterators and spliterators are *weakly consistent*.

Specified by:

navigableKeySet in interface ConcurrentNavigableMap<K,V>

Specified by:

navigableKeySet in interface NavigableMap<K,V>

Returns:

a navigable set view of the keys in this map

values

```
public Collection<V> values()
```

Returns a Collection view of the values contained in this map.

The collection's iterator returns the values in ascending order of the corresponding keys. The collections's spliterator additionally reports Spliterator.CONCURRENT, Spliterator.NONNULL and Spliterator.ORDERED, with an encounter order that is ascending order of the corresponding keys.

The collection is backed by the map, so changes to the map are reflected in the collection, and vice-versa. The collection supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`, `Collection.remove`, `removeAll`, `retainAll` and `clear` operations. It does not support the `add` or `addAll` operations.

The view's iterators and spliterators are *weakly consistent*.

Specified by:

`values` in interface `Map<K,V>`

Specified by:

`values` in interface `SortedMap<K,V>`

Overrides:

`values` in class `AbstractMap<K,V>`

Returns:

a collection view of the values contained in this map

entrySet

```
public Set<Map.Entry<K,V>> entrySet()
```

Returns a `Set` view of the mappings contained in this map.

The set's iterator returns the entries in ascending key order. The set's spliterator additionally reports `Spliterator.CONCURRENT`, `Spliterator.NONNULL`, `Spliterator.SORTED` and `Spliterator.ORDERED`, with an encounter order that is ascending key order.

The set is backed by the map, so changes to the map are reflected in the set, and vice-versa. The set supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`, `Set.remove`, `removeAll`, `retainAll` and `clear` operations. It does not support the `add` or `addAll` operations.

The view's iterators and spliterators are *weakly consistent*.

The `Map.Entry` elements traversed by the `iterator` or `spliterator` do *not* support the `setValue` operation.

Specified by:

`entrySet` in interface `Map<K,V>`

Specified by:

`entrySet` in interface `SortedMap<K,V>`

Returns:

a set view of the mappings contained in this map, sorted in ascending key order

descendingMap

```
public ConcurrentNavigableMap<K,V> descendingMap()
```

Description copied from interface: ConcurrentNavigableMap

Returns a reverse order view of the mappings contained in this map. The descending map is backed by this map, so changes to the map are reflected in the descending map, and vice-versa.

The returned map has an ordering equivalent to `Collections.reverseOrder(comparator())`. The expression `m.descendingMap().descendingMap()` returns a view of `m` essentially equivalent to `m`.

Specified by:

`descendingMap` in interface `ConcurrentNavigableMap<K,V>`

Specified by:

`descendingMap` in interface `NavigableMap<K,V>`

Returns:

a reverse order view of this map

descendingKeySet

```
public NavigableSet<K> descendingKeySet()
```

Description copied from interface: ConcurrentNavigableMap

Returns a reverse order `NavigableSet` view of the keys contained in this map. The set's iterator returns the keys in descending order. The set is backed by the map, so changes to the map are reflected in the set, and vice-versa. The set supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`, `Set.remove`, `removeAll`, `retainAll`, and `clear` operations. It does not support the `add` or `addAll` operations.

The view's iterators and spliterators are *weakly consistent*.

Specified by:

`descendingKeySet` in interface `ConcurrentNavigableMap<K,V>`

Specified by:

`descendingKeySet` in interface `NavigableMap<K,V>`

Returns:

a reverse order navigable set view of the keys in this map

equals

```
public boolean equals(Object o)
```

Compares the specified object with this map for equality. Returns `true` if the given object is also a map and the two maps represent the same mappings.

More formally, two maps `m1` and `m2` represent the same mappings if `m1.entrySet().equals(m2.entrySet())`. This operation may return misleading results if either map is concurrently modified during execution of this method.

Specified by:

`equals` in interface `Map<K,V>`

Overrides:

`equals` in class `AbstractMap<K,V>`

Parameters:

`o` - object to be compared for equality with this map

Returns:

`true` if the specified object is equal to this map

See Also:

`Object.hashCode()`, `HashMap`

putIfAbsent

```
public V putIfAbsent(K key,
                     V value)
```

If the specified key is not already associated with a value, associates it with the given value. This is equivalent to, for this `map`:

```
if (!map.containsKey(key))
  return map.put(key, value);
else
  return map.get(key);
```

except that the action is performed atomically.

Specified by:

`putIfAbsent` in interface `ConcurrentMap<K,V>`

Specified by:

`putIfAbsent` in interface `Map<K,V>`

Parameters:

`key` - key with which the specified value is to be associated

`value` - value to be associated with the specified key

Returns:

the previous value associated with the specified key, or `null` if there was no mapping for the key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key or value is null

remove

```
public boolean remove(Object key,
                      Object value)
```

Removes the entry for a key only if currently mapped to a given value. This is equivalent to, for this `map`:

```
if (map.containsKey(key)
    && Objects.equals(map.get(key), value)) {
  map.remove(key);
  return true;
} else {
  return false;
}
```

except that the action is performed atomically.

Specified by:

`remove` in interface `ConcurrentMap<K,V>`

Specified by:

`remove` in interface `Map<K,V>`

Parameters:

`key` - key with which the specified value is associated

`value` - value expected to be associated with the specified key

Returns:

`true` if the value was removed

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key is null

replace

```
public boolean replace(K key,
                       V oldValue,
                       V newValue)
```

Replaces the entry for a key only if currently mapped to a given value. This is equivalent to, for this `map`:

```
if (map.containsKey(key)
    && Objects.equals(map.get(key), oldValue)) {
  map.put(key, newValue);
  return true;
} else {
  return false;
}
```

except that the action is performed atomically.

Specified by:

`replace` in interface `ConcurrentMap<K,V>`

Specified by:

`replace` in interface `Map<K,V>`

Parameters:

`key` - key with which the specified value is associated

`oldValue` - value expected to be associated with the specified key

`newValue` - value to be associated with the specified key

Returns:

`true` if the value was replaced

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if any of the arguments are null

replace

```
public V replace(K key,
                 V value)
```

Replaces the entry for a key only if currently mapped to some value. This is equivalent to, for this `map`:

```
if (map.containsKey(key))
  return map.put(key, value);
else
  return null;
```

except that the action is performed atomically.

Specified by:

`replace` in interface `ConcurrentMap<K,V>`

Specified by:

`replace` in interface `Map<K,V>`

Parameters:

`key` - key with which the specified value is associated

`value` - value to be associated with the specified key

Returns:

the previous value associated with the specified key, or `null` if there was no mapping for the key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key or value is null

comparator

```
public Comparator<? super K> comparator()
```

Description copied from interface: `SortedMap`

Returns the comparator used to order the keys in this map, or `null` if this map uses the natural ordering of its keys.

Specified by:

`comparator` in interface `SortedMap<K,V>`

Returns:

the comparator used to order the keys in this map, or `null` if this map uses the natural ordering of its keys

firstKey

```
public K firstKey()
```

Description copied from interface: `SortedMap`

Returns the first (lowest) key currently in this map.

Specified by:

`firstKey` in interface `SortedMap<K,V>`

Returns:

the first (lowest) key currently in this map

Throws:

NoSuchElementException - if this map is empty

lastKey

```
public K lastKey()
```

Description copied from interface: SortedMap

Returns the last (highest) key currently in this map.

Specified by:

lastKey in interface SortedMap<K,V>

Returns:

the last (highest) key currently in this map

Throws:

NoSuchElementException - if this map is empty

putFirst

```
public V putFirst(K k,
                  V v)
```

Throws UnsupportedOperationException. The encounter order induced by this map's comparison method determines the position of mappings, so explicit positioning is not supported.

Specified by:

putFirst in interface SequencedMap<K,V>

Specified by:

putFirst in interface SortedMap<K,V>

Parameters:

k - the key

v - the value

Returns:

the value previously associated with k, or null if none

Throws:

UnsupportedOperationException - always

Since:

21

putLast

```
public V putLast(K k,
                 V v)
```

Throws `UnsupportedOperationException`. The encounter order induced by this map's comparison method determines the position of mappings, so explicit positioning is not supported.

Specified by:

`putLast` in interface `SequencedMap<K,V>`

Specified by:

`putLast` in interface `SortedMap<K,V>`

Parameters:

k - the key

v - the value

Returns:

the value previously associated with k, or null if none

Throws:

`UnsupportedOperationException` - always

Since:

21

subMap

```
public ConcurrentNavigableMap<K,V> subMap(K fromKey,
                                          boolean fromInclusive,
                                          K toKey,
                                          boolean toInclusive)
```

Description copied from interface: `NavigableMap`

Returns a view of the portion of this map whose keys range from `fromKey` to `toKey`. If `fromKey` and `toKey` are equal, the returned map is empty unless `fromInclusive` and `toInclusive` are both true. The returned map is backed by this map, so changes in the returned map are reflected in this map, and vice-versa. The returned map supports all optional map operations that this map supports.

The returned map will throw an `IllegalArgumentException` on an attempt to insert a key outside of its range, or to construct a submap either of whose endpoints lie outside its range.

Specified by:

`subMap` in interface `ConcurrentNavigableMap<K,V>`

Specified by:

subMap in interface NavigableMap<K,V>

Parameters:

fromKey - low endpoint of the keys in the returned map

fromInclusive - true if the low endpoint is to be included in the returned view

toKey - high endpoint of the keys in the returned map

toInclusive - true if the high endpoint is to be included in the returned view

Returns:

a view of the portion of this map whose keys range from fromKey to toKey

Throws:

ClassCastException - if fromKey and toKey cannot be compared to one another using this map's comparator (or, if the map has no comparator, using natural ordering). Implementations may, but are not required to, throw this exception if fromKey or toKey cannot be compared to keys currently in the map.

NullPointerException - if fromKey or toKey is null

IllegalArgumentException - if fromKey is greater than toKey; or if this map itself has a restricted range, and fromKey or toKey lies outside the bounds of the range

headMap

```
public ConcurrentNavigableMap<K,V> headMap(K toKey,
                                           boolean inclusive)
```

Description copied from interface: NavigableMap

Returns a view of the portion of this map whose keys are less than (or equal to, if inclusive is true) toKey. The returned map is backed by this map, so changes in the returned map are reflected in this map, and vice-versa. The returned map supports all optional map operations that this map supports.

The returned map will throw an IllegalArgumentException on an attempt to insert a key outside its range.

Specified by:

headMap in interface ConcurrentNavigableMap<K,V>

Specified by:

headMap in interface NavigableMap<K,V>

Parameters:

`toKey` - high endpoint of the keys in the returned map

`inclusive` - `true` if the high endpoint is to be included in the returned view

Returns:

a view of the portion of this map whose keys are less than (or equal to, if `inclusive` is true) `toKey`

Throws:

`ClassCastException` - if `toKey` is not compatible with this map's comparator (or, if the map has no comparator, if `toKey` does not implement `Comparable`). Implementations may, but are not required to, throw this exception if `toKey` cannot be compared to keys currently in the map.

`NullPointerException` - if `toKey` is null

`IllegalArgumentException` - if this map itself has a restricted range, and `toKey` lies outside the bounds of the range

tailMap

```
public ConcurrentNavigableMap<K,V> tailMap(K fromKey,
                                           boolean inclusive)
```

Description copied from interface: `NavigableMap`

Returns a view of the portion of this map whose keys are greater than (or equal to, if `inclusive` is true) `fromKey`. The returned map is backed by this map, so changes in the returned map are reflected in this map, and vice-versa. The returned map supports all optional map operations that this map supports.

The returned map will throw an `IllegalArgumentException` on an attempt to insert a key outside its range.

Specified by:

`tailMap` in interface `ConcurrentNavigableMap<K,V>`

Specified by:

`tailMap` in interface `NavigableMap<K,V>`

Parameters:

`fromKey` - low endpoint of the keys in the returned map

`inclusive` - `true` if the low endpoint is to be included in the returned view

Returns:

a
view of the portion of this map whose keys are greater than (or equal to, if `inclusive` is true) `fromKey`

Throws:

ClassCastException - if fromKey is not compatible with this map's comparator (or, if the map has no comparator, if fromKey does not implement Comparable). Implementations may, but are not required to, throw this exception if fromKey cannot be compared to keys currently in the map.

NullPointerException - if fromKey is null

IllegalArgumentException - if this map itself has a restricted range, and fromKey lies outside the bounds of the range

subMap

```
public ConcurrentNavigableMap<K,V> subMap(K fromKey,
                                          K toKey)
```

Description copied from interface: NavigableMap

Returns a view of the portion of this map whose keys range from fromKey, inclusive, to toKey, exclusive. (If fromKey and toKey are equal, the returned map is empty.) The returned map is backed by this map, so changes in the returned map are reflected in this map, and vice-versa. The returned map supports all optional map operations that this map supports.

The returned map will throw an IllegalArgumentException on an attempt to insert a key outside its range.

Equivalent to subMap(fromKey, true, toKey, false).

Specified by:

subMap in interface ConcurrentNavigableMap<K,V>

Specified by:

subMap in interface NavigableMap<K,V>

Specified by:

subMap in interface SortedMap<K,V>

Parameters:

fromKey - low endpoint (inclusive) of the keys in the returned map

toKey - high endpoint (exclusive) of the keys in the returned map

Returns:

a view of the portion of this map whose keys range from fromKey, inclusive, to toKey, exclusive

Throws:

ClassCastException - if fromKey and toKey cannot be compared to one another using this map's comparator (or, if the map has no comparator, using natural ordering). Implementations may, but are not required to,

throw this exception if `fromKey` or `toKey` cannot be compared to keys currently in the map.

`NullPointerException` - if `fromKey` or `toKey` is null

`IllegalArgumentException` - if `fromKey` is greater than `toKey`; or if this map itself has a restricted range, and `fromKey` or `toKey` lies outside the bounds of the range

headMap

```
public ConcurrentNavigableMap<K,V> headMap(K toKey)
```

Description copied from interface: `NavigableMap`

Returns a view of the portion of this map whose keys are strictly less than `toKey`. The returned map is backed by this map, so changes in the returned map are reflected in this map, and vice-versa. The returned map supports all optional map operations that this map supports.

The returned map will throw an `IllegalArgumentException` on an attempt to insert a key outside its range.

Equivalent to `headMap(toKey, false)`.

Specified by:

`headMap` in interface `ConcurrentNavigableMap<K,V>`

Specified by:

`headMap` in interface `NavigableMap<K,V>`

Specified by:

`headMap` in interface `SortedMap<K,V>`

Parameters:

`toKey` - high endpoint (exclusive) of the keys in the returned map

Returns:

a view of the portion of this map whose keys are strictly less than `toKey`

Throws:

`ClassCastException` - if `toKey` is not compatible with this map's comparator (or, if the map has no comparator, if `toKey` does not implement `Comparable`). Implementations may, but are not required to, throw this exception if `toKey` cannot be compared to keys currently in the map.

`NullPointerException` - if `toKey` is null

`IllegalArgumentException` - if this map itself has a restricted range, and `toKey` lies outside the bounds of the range

tailMap

```
public ConcurrentNavigableMap<K,V> tailMap(K fromKey)
```

Description copied from interface: NavigableMap

Returns a view of the portion of this map whose keys are greater than or equal to `fromKey`. The returned map is backed by this map, so changes in the returned map are reflected in this map, and vice-versa. The returned map supports all optional map operations that this map supports.

The returned map will throw an `IllegalArgumentException` on an attempt to insert a key outside its range.

Equivalent to `tailMap(fromKey, true)`.

Specified by:

`tailMap` in interface `ConcurrentNavigableMap<K,V>`

Specified by:

`tailMap` in interface `NavigableMap<K,V>`

Specified by:

`tailMap` in interface `SortedMap<K,V>`

Parameters:

`fromKey` - low endpoint (inclusive) of the keys in the returned map

Returns:

a view of the portion of this map whose keys are greater than or equal to `fromKey`

Throws:

`ClassCastException` - if `fromKey` is not compatible with this map's comparator (or, if the map has no comparator, if `fromKey` does not implement `Comparable`). Implementations may, but are not required to, throw this exception if `fromKey` cannot be compared to keys currently in the map.

`NullPointerException` - if `fromKey` is null

`IllegalArgumentException` - if this map itself has a restricted range, and `fromKey` lies outside the bounds of the range

lowerEntry

```
public Map.Entry<K,V> lowerEntry(K key)
```

Returns a key-value mapping associated with the greatest key strictly less than the given key, or `null` if there is no such key. The returned entry does *not* support the `Entry.setValue` method.

Specified by:

`lowerEntry` in interface `NavigableMap<K,V>`

Parameters:

key - the key

Returns:

an entry with the greatest key less than `key`, or `null` if there is no such key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key is null

lowerKey

```
public K lowerKey(K key)
```

Description copied from interface: `NavigableMap`

Returns the greatest key strictly less than the given key, or `null` if there is no such key.

Specified by:

`lowerKey` in interface `NavigableMap<K,V>`

Parameters:

key - the key

Returns:

the greatest key less than `key`, or `null` if there is no such key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key is null

floorEntry

```
public Map.Entry<K,V> floorEntry(K key)
```

Returns a key-value mapping associated with the greatest key less than or equal to the given key, or `null` if there is no such key. The returned entry does *not* support the `Entry.setValue` method.

Specified by:

`floorEntry` in interface `NavigableMap<K,V>`

Parameters:

key - the key

Returns:

an entry with the greatest key less than or equal to `key`, or `null` if there is no such key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key is null

floorKey

```
public K floorKey(K key)
```

Description copied from interface: `NavigableMap`

Returns the greatest key less than or equal to the given key, or `null` if there is no such key.

Specified by:

`floorKey` in interface `NavigableMap<K,V>`

Parameters:

key - the key

Returns:

the greatest key less than or equal to `key`, or `null` if there is no such key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key is null

ceilingEntry

```
public Map.Entry<K,V> ceilingEntry(K key)
```

Returns a key-value mapping associated with the least key greater than or equal to the given key, or `null` if there is no such entry. The returned entry does *not* support the `Entry.setValue` method.

Specified by:

`ceilingEntry` in interface `NavigableMap<K,V>`

Parameters:

key - the key

Returns:

an entry with the least key greater than or equal to `key`, or `null` if there is no such key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key is null

ceilingKey

```
public K ceilingKey(K key)
```

Description copied from interface: `NavigableMap`

Returns the least key greater than or equal to the given key, or `null` if there is no such key.

Specified by:

`ceilingKey` in interface `NavigableMap<K,V>`

Parameters:

`key` - the key

Returns:

the least key greater than or equal to `key`, or `null` if there is no such key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key is null

higherEntry

```
public Map.Entry<K,V> higherEntry(K key)
```

Returns a key-value mapping associated with the least key strictly greater than the given key, or `null` if there is no such key. The returned entry does *not* support the `Entry.setValue` method.

Specified by:

`higherEntry` in interface `NavigableMap<K,V>`

Parameters:

`key` - the key

Returns:

an entry with the least key greater than `key`, or `null` if there is no such key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key is null

higherKey

`public K higherKey(K key)`

Description copied from interface: `NavigableMap`

Returns the least key strictly greater than the given key, or `null` if there is no such key.

Specified by:

`higherKey` in interface `NavigableMap<K,V>`

Parameters:

`key` - the key

Returns:

the least key greater than `key`, or `null` if there is no such key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key is null

firstEntry

`public Map.Entry<K,V> firstEntry()`

Returns a key-value mapping associated with the least key in this map, or `null` if the map is empty. The returned entry does *not* support the `Entry.setValue` method.

Specified by:

`firstEntry` in interface `NavigableMap<K,V>`

Specified by:

`firstEntry` in interface `SequencedMap<K,V>`

Returns:

an entry with the least key, or `null` if this map is empty

lastEntry

`public Map.Entry<K,V> lastEntry()`

Returns a key-value mapping associated with the greatest key in this map, or `null` if the map is empty. The returned entry does *not* support the `Entry.setValue` method.

Specified by:

`lastEntry` in interface `NavigableMap<K,V>`

Specified by:

`lastEntry` in interface `SequencedMap<K,V>`

Returns:

an entry with the greatest key, or `null` if this map is empty

pollFirstEntry

```
public Map.Entry<K,V> pollFirstEntry()
```

Removes and returns a key-value mapping associated with the least key in this map, or `null` if the map is empty. The returned entry does *not* support the `Entry.setValue` method.

Specified by:

`pollFirstEntry` in interface `NavigableMap<K,V>`

Specified by:

`pollFirstEntry` in interface `SequencedMap<K,V>`

Returns:

the removed first entry of this map, or `null` if this map is empty

pollLastEntry

```
public Map.Entry<K,V> pollLastEntry()
```

Removes and returns a key-value mapping associated with the greatest key in this map, or `null` if the map is empty. The returned entry does *not* support the `Entry.setValue` method.

Specified by:

`pollLastEntry` in interface `NavigableMap<K,V>`

Specified by:

`pollLastEntry` in interface `SequencedMap<K,V>`

Returns:

the removed last entry of this map, or `null` if this map is empty

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.